BEYOND SPEECH GENERATION

Toward Thread-Aware Conversational Intelligence in Large Language Models

A framework for managing, pausing and resuming multiple interleaved conversational threads while preserving reasoning continuity

RESEARCH PROPOSAL / CONCEPTUAL FRAMEWORK

Version 1.0 | 3 November 2026

Pranav Patil

Independent Researcher

LinkedIn: https://www.linkedin.com/in/pranav-patil-78825621b/

PRE-PROTOTYPE DOCUMENT • NO EXPERIMENTAL RESULTS ARE CLAIMED

Document status

This manuscript is a research proposal and conceptual framework. The prototype has not yet been implemented, and no experiments have been run. The architecture, hypotheses, benchmark design and evaluation metrics below are proposed research contributions, not validated findings. Any claim of novelty is provisional and must be tested through a systematic literature review and comparison with existing systems.

Scope note The paper focuses on thread continuity and answer resumption. Humour and broader social pragmatics are acknowledged as important dimensions of natural conversation, but they are outside the primary scope of this first study. Including them as a second central problem would make the research question less testable.

Contents

- Abstract and keywords
- 1. Introduction
- 2. Background and related work
- 3. Problem formulation
- 4. Research questions and hypotheses
- 5. Proposed framework: Thread-Aware Conversational Architecture
- 6. Illustrative interaction scenarios
- 7. Experimental methodology and evaluation plan
- 8. Discussion: implications and trade-offs
- 9. Limitations and future work
- 10. Conclusion
- References
- Appendix A. Proposed thread-state schema
- Appendix B. Benchmark annotation taxonomy
- Appendix C. High-level interaction algorithm

Abstract

Human conversation is not strictly linear. A participant can begin answering a complex question, respond to a brief clarification or an unrelated side question, incorporate information introduced during that exchange, and then continue the original discussion without reconstructing the conversation from the beginning. Many conversational AI interfaces, by contrast, organise interaction as a sequence of completed user turns and assistant turns. When a user interrupts or redirects an assistant response, the system may prioritise the latest message while failing to preserve the unfinished structure of the earlier answer. This creates a continuity problem that is distinct from ordinary long-context recall: the system must retain not only what was said, but also what the suspended conversational task was trying to accomplish, what portions of its answer have already been delivered, what remains unresolved, and how a detour may affect the original task.

This paper proposes Thread-Aware Conversational Architecture (TACA), a framework for managing multiple interleaved conversational threads during text and real-time voice interaction.

TACA combines streaming interruption detection, intent and thread-relation classification, a graph-structured thread manager, structured answer checkpoints, and a resumption policy. A checkpoint records task-level state, including the original goal, assumptions, completed answer units, unresolved requirements, evidence introduced, and the next planned response unit; it does not require storage or disclosure of a model's private chain-of-thought. When a user interjects, the system pauses or revises the current response, determines whether the input is a clarification, side question, additional context, correction, or topic change, and routes it accordingly. After answering a side question, the system can resume the suspended thread and integrate newly acquired information when it is relevant to the original objective.

The paper formalises the problem, reviews related work in dialogue state tracking, topic-shift modelling, conversational memory, full-duplex speech interaction, and interruption evaluation, and outlines a benchmark and experimental methodology. The prototype and experiments remain future work. The proposed contribution is therefore a testable research framework, not a claim of demonstrated performance. The central hypothesis is that explicit thread-state representation and answer-progress checkpoints can improve thread identification, reasoning continuity, objective preservation, and resumption accuracy relative to linear-dialogue and transcript-only baselines.

Keywords: conversational AI; large language models; dialogue management; multi-threaded dialogue; conversational state tracking; full-duplex speech; interruption handling; reasoning continuity; response resumption; dialogue benchmarks.

1. Introduction

Human conversation supports overlapping intentions and temporary detours. A person may be explaining a complex idea when a listener asks for the meaning of one term, requests a quick fact, adds a constraint, or changes a detail. The speaker can respond to that local contribution and continue the larger explanation. The conversation therefore behaves less like a single queue of independent prompts and more like a changing set of related tasks with different levels of priority.

A simple example illustrates the problem. A user asks how humanity could build a city on Mars. An assistant begins discussing habitat design, energy, food production, transport and environmental constraints. During the explanation, the user asks, “What is the temperature on Mars?” A useful conversational system should answer the immediate question, then resume the larger city-planning explanation. If the temperature information affects habitat insulation, energy demand or outdoor operations, the resumed answer should explicitly connect that new information to the original plan. The side question is not a replacement for the main question; it is a temporary branch that may enrich it.

A conventional turn-based interface can lose this structure. When a new user message arrives, the system may treat it as the only current task. The earlier response may be truncated, displaced or forgotten as an unfinished activity. Even when the full transcript remains available, the model may not know exactly where its answer stopped, which parts were already covered, which requirements remain unanswered, or whether the new information should be incorporated into the parent task. Transcript retention is therefore not equivalent to task continuity.

The distinction matters increasingly in voice interaction. Full-duplex systems can listen while speaking, making mid-response user input technically possible. Yet the ability to receive overlapping speech does not automatically provide a mechanism for preserving and resuming the

assistant's unfinished answer. The system must coordinate acoustic interruption, semantic interpretation, conversational control, thread state and response planning.

1.1 Research motivation

This work is motivated by five related capabilities that should be treated as one coordinated problem:

- Interruptible generation: the user can speak or type while an assistant response is being produced, and the system can safely pause or revise its output.
- Intent-sensitive routing: the system distinguishes a clarification, a side question, a correction, an additional constraint, a continuation and a permanent topic change.
- Explicit thread state: the system maintains separate but connected records for unfinished conversational tasks.
- Answer-progress preservation: the system records what the current answer has already delivered and what remains to be addressed.
- Relevance-aware resumption: after a detour, the system resumes the appropriate task and integrates information from the detour only when it is relevant.

1.2 Research objective

The objective is to design and evaluate a framework that allows a conversational LLM to manage multiple active or suspended threads while preserving the progress of an unfinished response. The initial study will focus on complex explanatory and problem-solving tasks that can be evaluated for coverage, continuity and correctness. Voice interaction is an important target modality, but the underlying thread-management problem can first be isolated and tested in text before adding the additional complexity of live audio.

1.3 Intended contributions

1. A formal problem definition for interleaved conversational threads and unfinished-answer resumption.
2. A proposed architecture combining thread graphs, structured answer checkpoints, interruption handling and context-aware resumption.
3. A proposed benchmark and evaluation protocol for measuring thread identification, resumption accuracy, continuity, objective preservation and unnecessary resumption.
4. An explicit analysis of deployment trade-offs, including latency, memory, ambiguity, privacy and user control.

These are intended contributions. Their novelty and value must be established through implementation, baseline comparison and independent evaluation.

2. Background and related work

The proposed problem intersects several research areas. Existing work has addressed parts of the problem, but the specific combination of live interruption, explicit management of suspended tasks, answer-progress checkpoints and context-aware continuation needs to be examined directly. The following review is a focused starting point rather than an exhaustive systematic review.

2.1 Dialogue segmentation and state tracking

Dialogue state tracking (DST) represents the evolving state of a conversation, while dialogue segmentation identifies boundaries between topics or conversational episodes. Das et al. introduced S3-DST, which combines open-domain dialogue segmentation and state tracking for LLM-era conversations [1]. The work is especially relevant because open-domain interactions can include multiple topics and tasks rather than a single constrained service flow. It demonstrates that dialogue orchestration requires more than storing an undifferentiated transcript. For the present proposal, the remaining question is whether a state representation can also preserve the progress of an assistant response that was interrupted before completion, then support its resumption after a side exchange.

S3-DST provides a strong conceptual foundation for identifying topic segments and tracking their state. TACA would extend the unit of management from a topic segment to an unfinished conversational task with a parent-child relationship to temporary side threads. The difference is a research hypothesis to be tested, not an established superiority claim.

2.2 Topic shifts and inter-topic transitions

Hwang et al. proposed MP2D, a framework for generating dialogue with natural topic transitions, and introduced TS-WikiDialog as a topic-shift dialogue benchmark [2]. Their work reports that LLMs can struggle with topic shifts and that training data designed for transitions can improve performance. This is directly relevant to a system that must recognise when a conversation temporarily moves away from its current subject.

However, a topic shift is not always a permanent change of task. In the Mars example, the temperature question is a detour that remains relevant to the main city-design task. TACA therefore distinguishes a temporary side thread from a permanent topic change and represents an explicit relation between them. A successful system should not indiscriminately resume every earlier topic; it should infer whether a suspended task remains active and whether the user intends to return to it.

2.3 Long-term conversational memory

Maharana et al. introduced LoCoMo, a benchmark for evaluating very long-term conversational memory across conversations that can span hundreds of turns and multiple sessions [3]. Their analysis shows that long-context models and retrieval-augmented strategies still face challenges with long-range temporal and causal information. This work motivates structured memory and careful evaluation of what a model retains over extended interaction.

The target of TACA is narrower than general long-term memory but more process-oriented. It is not sufficient to retrieve a fact from the past. The system must preserve an unfinished task's current state and next response obligation. The proposed checkpoint is therefore a task-progress representation, not merely a conversational summary or user profile.

2.4 Full-duplex speech interaction and turn-taking

Défossez et al. presented Moshi, a speech-text foundation model and full-duplex spoken dialogue framework designed to model user and assistant audio streams in parallel [4]. The paper identifies limitations of conventional cascaded pipelines, including latency, the loss of paralinguistic information through text-only intermediate representations, and poor treatment of interruptions

and overlapping speech. Moshi demonstrates that real-time speech interaction can be modelled beyond a strictly sequential listen-then-speak pipeline.

Arora et al. introduced Talking Turns, a benchmark for evaluating the turn-taking behaviour of audio foundation models [5]. Their results show that systems can struggle to decide when to speak, can interrupt too aggressively, and may rarely provide backchannels. The work establishes that turn-taking is a measurable conversational capability rather than a simple consequence of speech synthesis quality.

These studies support the feasibility and importance of listening while speaking. They do not, by themselves, solve the task-level question addressed here: after a user interjects, what structured state should be preserved so that the assistant can answer the interjection and continue the interrupted response from an appropriate point?

2.5 Natural multi-turn audio interaction and interruption evaluation

Gosai et al. introduced Audio MultiChallenge, a multi-turn benchmark built from natural speech conversations. Its evaluation includes instruction retention, inference memory, self-coherence and a voice-editing dimension covering mid-utterance repairs and backtracking [6]. The authors report persistent difficulties for current systems under natural speech conditions, including corrections and non-monotonic phrasing. This benchmark is particularly relevant for evaluating the robustness of an audio-native implementation.

Kuan et al. introduced Model Interruption Bench (MIB), which evaluates real-time content-aware interruption in full-duplex voice agents [7]. The benchmark studies whether a voice agent can interrupt spoken reasoning at an appropriate moment to correct an error. Its findings highlight a gap between recognising an error in a turn-based setting and acting on it during live speech. This underscores that real-time interruption is a separate systems challenge from offline understanding.

MIB focuses on proactive assistant interruption to correct a user's content, while the present proposal focuses primarily on user-initiated interruption of an assistant response, followed by handling a side task and resuming the parent response. Both problems involve full-duplex interaction and timing, but their control direction and task-level objective differ.

2.6 Related-work synthesis and candidate research gap

Area	Representative work	What it contributes	Question left for this proposal
Dialogue segmentation and state tracking	S3-DST [1]	Segments open-domain dialogue and tracks conversational state.	Can state tracking preserve an interrupted answer's progress and next unresolved unit?
Topic transitions	MP2D / TS-WikiDialog [2]	Generates and evaluates dialogues with topic shifts.	Can the system distinguish temporary detours from permanent topic changes and maintain parent-child links?
Long-term memory	LoCoMo [3]	Evaluates recall and understanding over long conversations.	Can the system preserve procedural progress, not only facts and events?

Area	Representative work	What it contributes	Question left for this proposal
Full-duplex speech	Moshi [4]	Supports low-latency, simultaneous listening and speaking.	How should a live interruption affect the suspended answer plan and its later resumption?
Turn-taking	Talking Turns [5]	Measures turn-taking behaviour in audio foundation models.	Can turn-taking decisions be coordinated with multi-thread task management?
Natural multi-turn audio	Audio MultiChallenge [6]	Evaluates voice edits, backtracking, memory and coherence.	Can benchmark scenarios isolate interleaved side questions followed by structured answer resumption?
Real-time interruption	Model Interruption Bench [7]	Measures content-aware interruption in full-duplex agents.	Can user-initiated interruptions create a temporary side thread without losing the parent response?

Novelty boundary The literature already covers dialogue state tracking, topic shifts, long-term memory, full-duplex speech, turn-taking and real-time interruption. The candidate gap is their integration around a specific capability: preserving the structured progress of an unfinished assistant answer, managing an interleaved side thread, and resuming the original answer with relevance-aware incorporation of information from the detour. A systematic search is required before claiming that this exact combination is novel.

3. Problem formulation

3.1 Definitions

Term	Operational definition
Conversational thread	A coherent user goal or subtask that may span several turns and may remain unfinished while another thread is handled.
Parent thread	The main task that is temporarily suspended while a related or unrelated side task is addressed.
Side thread	A temporary question, clarification or subtask initiated during a parent thread.
Thread relation	A typed link between threads, such as clarification-of, derived-from, adds-evidence-to, corrects, or unrelated-detour-from.
Answer checkpoint	A compact, structured record of the assistant's emitted progress and remaining response obligations for a particular thread.

Term	Operational definition
Resumption	Continuation of a suspended thread from its next unresolved response unit, using any relevant new context acquired during the detour.
Permanent topic change	A user-directed transition where the previous thread is not expected to resume unless the user later requests it.

3.2 Conversation model

Let a conversation be an event sequence E = (e1, e2, ..., eT), where each event may be a user utterance, assistant output segment, control event (for example, stop or resume), or tool result. Let Gt = (Vt, Rt) be a thread graph at time t. Each node Ti in Vt represents a conversational thread, and each relation in Rt records how threads are connected. A thread can be active, suspended, completed, abandoned or awaiting clarification.

A thread state is represented as:

```
T_i = (g_i, c_i, a_i, d_i, u_i, e_i, n_i, s_i)
```

where:

- g_i: the user's goal or task objective;
- c_i: relevant conversational context and linked evidence;
- a_i: explicit assumptions and constraints;
- d_i: completed answer units or requirements already delivered;
- u_i: unresolved questions, requirements or answer units;
- e_i: relevant evidence, facts or tool outputs introduced so far;
- n_i: the next planned response unit or action;
- s_i: lifecycle status (active, suspended, completed, abandoned or clarification-needed).

The state is deliberately task-oriented. It should not be implemented by storing a purported verbatim private chain-of-thought. A system can preserve user-visible claims, answer structure, assumptions, evidence references and outstanding requirements without exposing or storing hidden internal reasoning traces.

3.3 Incoming-event classification

For each incoming user event x_t, the system estimates an intent label z_t and a target thread k_t. The proposed intent set is:

Intent class	Meaning	Expected state action
Continue	The user is continuing the current task.	Append information and update the active thread.
Clarification	The user asks about a term or part of the current answer.	Create or mark a short clarification subtask; preserve the parent answer.
Side question	A temporary question, related or unrelated to the parent goal.	Create a child or sibling thread and suspend the parent.

Intent class	Meaning	Expected state action
Additional context	The user adds facts, preferences or constraints.	Update the relevant thread; revise the remaining answer plan.
Correction	The user says an earlier assumption or statement is wrong.	Invalidate affected state and re-plan dependent answer units.
Topic change	The user chooses a new subject or goal.	Mark the former thread suspended or abandoned according to intent.
Control event	Stop, pause, resume, repeat or speak more slowly.	Apply the requested control action.
Ambiguous	The correct thread or intention is uncertain.	Ask a concise clarification or use a reversible low-risk action.

3.4 Policy and objective

The controller selects an action from a policy $\pi(a_t \mid x_t, G_t, q_t)$, where q_t is the current generation and playback state. Candidate actions include continue, pause, checkpoint, create-side-thread, update-parent, answer-side-thread, resume-parent, abandon-thread and ask-clarification. The policy should consider both semantic intent and interaction timing.

A proposed optimisation objective is to minimise a weighted conversational cost:

$$J(\pi) = \lambda_r L_{routing} + \lambda_c L_{continuity} + \lambda_g L_{goal\text{-}loss} + \lambda_x L_{context\text{-}integration} + \lambda_f L_{false\text{-}resumption} + \lambda_l L_{latency} + \lambda_p L_{privacy\text{-}and\text{-}compute}$$

The terms represent routing errors, loss of answer continuity, failure to satisfy the original goal, incorrect integration of side-thread information, unwanted resumption, interaction delay, and privacy or computational cost. The weights λ are design parameters that must be calibrated through evaluation. This formulation is a research objective, not a claim that these losses can already be measured perfectly.

4. Research questions and hypotheses

1. RQ1: Can explicit thread-state tracking improve the identification and management of temporary side questions compared with a linear transcript-only dialogue baseline?
2. RQ2: Do structured answer-progress checkpoints improve the accuracy and coherence of resuming an interrupted complex answer?
3. RQ3: Can the system correctly integrate information from a side thread when it is relevant to the parent task, while avoiding forced or irrelevant connections?
4. RQ4: How do interruption timing, number of active threads, detour length and ambiguity affect resumption performance?
5. RQ5: What latency, memory and privacy trade-offs arise when thread management is combined with live, full-duplex voice interaction?

4.1 Hypotheses

- H1: A system with explicit thread tracking will achieve higher thread-identification accuracy than a transcript-only baseline on interleaved conversations.
- H2: Structured answer checkpoints will improve resumption-point accuracy and reduce redundant repetition relative to transcript summaries alone.
- H3: A relevance-aware continuation policy will improve completion of the original task without increasing the rate of irrelevant context insertion.
- H4: Performance will degrade as the number of concurrently active threads, detour length and ambiguity increase, but the degradation will be smaller for the proposed architecture than for the baselines.
- H5: Full-duplex input will reduce the interaction cost of user-initiated corrections and side questions, but will introduce additional latency, acoustic and privacy trade-offs.

5. Proposed framework: Thread-Aware Conversational Architecture (TACA)

TACA is a proposed orchestration layer around a conversational LLM and a streaming speech interface. It does not require a new foundation model as the first implementation. A prototype can use an existing text or speech model while testing whether explicit control and state management improve the interaction. The architecture separates perception, thread management, answer progress and response generation so that each component can be evaluated independently.

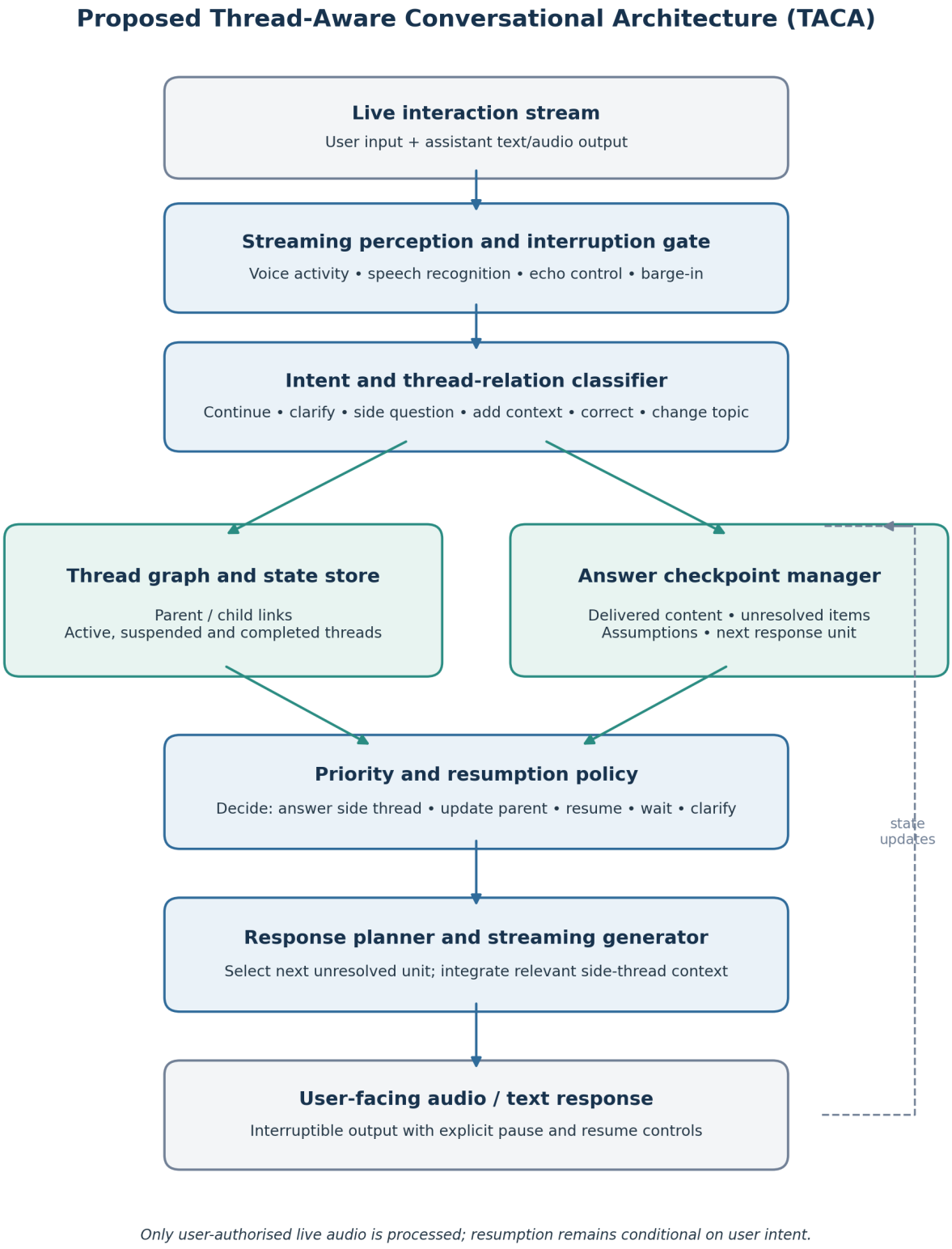


Figure 1. Proposed TACA architecture. The thread graph, checkpoint manager and resumption policy coordinate live interruption with task continuity.

5.1 Streaming perception and interruption gate

In voice mode, the microphone input remains available while the assistant is speaking only when the user has explicitly enabled a live, full-duplex interaction mode. A streaming perception module combines voice-activity detection, speech recognition, endpointing, speaker and echo handling,

and a barge-in detector. It should distinguish user speech from the assistant's own audio leaking back through the microphone. The system must support immediate stop commands, corrections and natural interjections without forcing the user to wait for a long generated response to finish.

The interruption gate decides whether to stop playback immediately, pause at the next safe audio boundary, or continue listening while buffering the utterance. A hard stop instruction should take precedence over a preference for sentence-boundary completion. For ordinary interjections, a short semantic boundary can reduce clipped speech, but the interaction policy must not delay urgent user control.

5.2 Intent and thread-relation classifier

The classifier receives a partial or complete user utterance, the current generation state and a compact view of active threads. It predicts both the communicative intent and the thread to which the utterance belongs. These are separate predictions: an utterance may be a clarification of the parent task, an unrelated side question, a correction that changes an assumption, or a new independent topic. The classifier should also estimate confidence and defer to a clarification question when the consequences of misrouting are high.

5.3 Thread graph and state store

Threads are represented as a graph rather than a flat list. A side question can be linked to a parent by relations such as "clarifies", "adds evidence to", "depends on", "corrects", or "temporarily branches from". An unrelated detour may be linked only by temporal sequence. The graph makes it possible to preserve the parent task while giving a side thread its own goal, answer and lifecycle status. State should be updated incrementally and versioned so that corrections can invalidate affected assumptions without destroying unrelated context.

5.4 Answer checkpoint manager

The checkpoint manager records the progress of the assistant's answer as it is streamed. It should capture: (a) the user's original objective; (b) the answer outline or required coverage units; (c) claims or sections already emitted; (d) unresolved parts; (e) assumptions and evidence already introduced; (f) the current output position; and (g) the next planned response unit. This is more useful for resumption than a generic summary such as "the assistant was explaining Mars city design".

A checkpoint must be robust to partial sentences, generated text that was never played, speech that was played but not transcribed correctly, and answer sections invalidated by a user correction. The implementation should distinguish generated, queued, played and acknowledged content. The resumption point should be based on what the user actually received, not merely what the model generated internally.

5.5 Side-thread processor

When a side question is detected, the system creates a temporary thread and answers it using the relevant context. The side thread may be independent, or it may contribute facts that affect the parent task. For example, a question about Mars temperature is directly relevant to habitat insulation and outdoor operations. A side question about the current time may not be relevant to the city-design explanation. TACA therefore stores the side-thread result and a typed relation, but does not assume that every detour should be woven into the parent answer.

5.6 Relevance-aware resumption planner

After the side thread is answered, the planner decides whether to resume the parent immediately, wait for a user cue, or ask whether the user wants to continue. The decision should depend on explicit user instructions, the inferred temporary nature of the detour, the system's confidence and the interface's configured behaviour. It then selects the next unresolved response unit and checks whether new evidence from the side thread changes the parent answer plan.

When a connection is relevant, the resumed answer should make the bridge explicit. For example: "Since the Martian environment is extremely cold and its atmosphere is thin, the habitat design needs strong thermal control and pressure protection. Returning to the city plan, this affects…" When the side question is unrelated, a simple continuation cue is better: "Returning to the Mars city plan, the next major issue is food production…" The system should avoid inventing a relationship merely to sound conversational.

5.7 Interruptible response generation

The response generator produces text and, in voice mode, audio in a stream. The orchestrator must be able to cancel or pause queued output, preserve the checkpoint, answer the new thread and restart generation from the saved task state. This does not require pausing the same model computation in place. Many model-serving APIs are stateless between requests; a practical implementation may instead stop the current generation, serialise a checkpoint and issue a new generation request conditioned on that checkpoint and the updated thread graph.

This distinction is important: TACA proposes logical continuity of the task, not guaranteed persistence of the model's hidden activations or private internal reasoning state. A reliable checkpoint should be interpretable, testable and portable across model calls.

5.8 User control, consent and privacy

A live microphone should not be treated as an unconditional always-on sensor. The prototype should use an explicit user-controlled live mode, a clear recording/listening indicator, an immediate mute control, and documented retention behaviour. Where feasible, voice-activity detection and other low-level processing should occur locally. Raw audio should not be retained by default unless the user explicitly opts in and the research protocol has appropriate consent. A voice system must also manage acoustic echo, bystander speech, accidental activation and misattributed speakers.

5.9 Proposed interaction lifecycle

1. The user starts a main task; the system creates a parent thread and an answer plan.
2. The assistant begins streaming a response while updating a checkpoint from the content actually delivered.
3. The user interjects; the interruption gate pauses or stops output and captures the utterance.
4. The classifier identifies whether the utterance is a clarification, side question, context addition, correction, control command or topic change.
5. The thread manager updates the graph; a temporary side thread is created when appropriate, while the parent remains suspended.
6. The assistant answers the side thread. Any information relevant to the parent is attached with provenance and a relation label.

7. The resumption policy decides whether to continue, wait, ask, or abandon the parent thread.
8. If resumed, the planner selects the next unresolved answer unit, generates a short context bridge when useful, and continues the original response.

6. Illustrative interaction scenarios

6.1 Related side question: Mars city design

Stage	Conversation	TACA state
Main task	User: “How could humanity build a city on Mars?” Assistant begins explaining habitat architecture, energy, food and transport.	Parent thread T1 active. Goal and answer outline stored; early sections marked delivered.
Interruption	User: “By the way, what is the temperature on Mars?”	T1 suspended at current checkpoint. Classifier labels the input as a side question related to T1.
Side answer	Assistant answers the temperature question using a reliable information source or model knowledge with appropriate uncertainty.	Child thread T2 created and answered. The result is linked to T1 as potentially relevant environmental context.
Resumption	Assistant: “That cold environment means the city needs robust thermal insulation and heat management. Returning to the city plan, this affects habitat design first, and then energy demand…”	TACA resumes T1 at the next unresolved unit and integrates T2 only where relevant.

6.2 Clarification during a technical explanation

User: “Explain how an AI could help discover a new material for room-temperature superconductivity.” The assistant begins with electronic structure and crystal-lattice properties. The user interrupts: “Wait, what does electronic structure mean?” The system treats this as a clarification thread linked to the current explanation, answers the definition, and resumes by connecting electronic structure to electron interactions and candidate material properties. The parent answer is not restarted from the beginning.

6.3 Additional information rather than a side question

During a travel-planning discussion, the user says: “I forgot to mention that I cannot travel on weekends.” This is not a side question. It is a constraint update that changes the parent task. The system should update the parent thread and revise the remaining plan rather than creating a detached side thread or answering the statement as a standalone question.

6.4 Permanent topic change

During the Mars discussion, the user says: “Forget the Mars project. Help me prepare for my programming exam instead.” The system should not automatically resume the Mars thread after answering the exam question. It should mark the former thread as suspended or abandoned according to user intent and resume it only when the user explicitly returns.

7. Experimental methodology and evaluation plan

Implementation status The prototype has not yet been built. This section specifies a proposed evaluation plan. No quantitative results, performance improvements or user-study findings are reported in this manuscript.

7.1 Prototype stages

1. Phase A - Text-only orchestration prototype: implement a thread graph, intent classifier, checkpoint schema and resumption planner around a fixed LLM. This isolates the core thread-continuity problem from speech-recognition and audio-latency issues.
2. Phase B - Streaming text interruption: allow a user message to arrive while text output is streaming; stop generation, checkpoint the answer and route the new message.
3. Phase C - Voice prototype: integrate streaming audio input, voice activity detection, echo handling, speech output and user-controlled barge-in. Evaluate acoustic robustness and latency separately from thread reasoning.

7.2 Benchmark: Interleaved Conversation Resumption Benchmark (ICRB)

A proposed benchmark, provisionally named the Interleaved Conversation Resumption Benchmark (ICRB), will evaluate whether a system can preserve and resume unfinished conversational tasks. ICRB is a planned dataset and protocol, not an existing released benchmark. Its design should complement rather than duplicate topic-shift, long-term-memory and audio-interaction benchmarks such as TS-WikiDialog, LoCoMo and Audio MultiChallenge [2, 3, 6].

Scenario family	Test design	Expected capability
Related detour	A brief factual question directly affects the parent task.	Answer the detour and integrate its result into the parent response.
Unrelated detour	A brief unrelated question interrupts a complex answer.	Answer the detour, then resume without forcing a false connection.
Clarification	The user asks for a definition or explanation of one component.	Clarify locally and continue the parent explanation.
Constraint addition	The user adds a new requirement while the answer is in progress.	Update the parent state and revise downstream content.
Correction	The user corrects an assumption or fact.	Invalidate dependent content and re-plan the response.
Multiple detours	Two or more side threads occur before resumption.	Maintain distinct thread states and choose the correct parent.
Competing parent tasks	More than one complex thread is suspended.	Resume the intended thread rather than the most recent one by default.
Delayed return	The user returns after many intervening turns.	Retrieve the right checkpoint and continue accurately.
No-resume instruction	The user explicitly changes topic or abandons the earlier task.	Avoid unwanted automatic resumption.

Scenario family	Test design	Expected capability
Mid-generation barge-in	The user speaks while assistant audio is playing.	Detect, capture, route and handle the interruption in real time.

7.3 Dataset construction and annotation

The initial benchmark should contain human-authored or carefully reviewed scenarios with controlled variations in task complexity, detour relation, interruption position and number of active threads. Synthetic generation may be used to expand coverage, but a subset must be independently reviewed by human annotators to prevent benchmark artefacts and unrealistic dialogue. Each example should include a thread graph, intent labels, the correct target thread, the parent task's completed and unresolved requirements, whether resumption is appropriate, and criteria for evaluating context integration.

A pilot set of approximately 100 scenarios is a reasonable starting point for debugging the taxonomy and annotation protocol, not a statistically justified final benchmark size. The full evaluation set should be sized after a pilot estimates variance and a power analysis defines the minimum effect of interest. Train, development and test splits should separate templates and underlying task content to reduce leakage.

7.4 Baselines and ablations

System	Description	Purpose
B0: Linear baseline	Standard turn-based LLM using the available conversation transcript.	Measures ordinary dialogue behaviour.
B1: Transcript + summary	LLM with periodic summaries of the ongoing conversation.	Tests whether summarisation alone is sufficient.
B2: Topic/state tracker	LLM with dialogue segmentation and explicit state fields, but no answer-progress checkpoint.	Tests the value of topic and state tracking.
B3: Thread graph only	LLM with active/suspended thread graph, but generic thread summaries.	Tests the value of explicit thread relations.
TACA full	Thread graph + answer-progress checkpoint + relevance-aware resumption policy.	Tests the complete proposed framework.

Ablation studies should remove one component at a time: no checkpoint, no parent-child links, no relevance test, no explicit user-intent confidence threshold, or no progress tracking during streaming. For fair comparison, the base model, tool access, context budget and response constraints should be held constant wherever possible.

7.5 Evaluation metrics

Metric	Definition / measurement
Thread identification accuracy (TIA)	Correct target-thread assignments divided by all annotated routing decisions.
Resumption-point accuracy (RPA)	Proportion of resumptions that continue from the correct next unresolved answer unit.
Parent-goal completion (PGC)	Proportion of required parent-task elements correctly completed by the end of the dialogue.
Answer continuity score (ACS)	Human or rubric-based score for logical flow between the pre-interruption and resumed answer.
Context-integration precision (CIP)	Proportion of inserted side-thread facts that are relevant, supported and correctly applied to the parent task.
Context-integration recall (CIR)	Proportion of relevant side-thread facts that are appropriately incorporated when needed.
Redundancy rate (RR)	Repeated answer content that was already delivered and remains valid.
False-resumption rate (FRR)	Rate at which a system resumes a thread after the user intended a permanent topic change or no return.
Unresolved-obligation rate (UOR)	Fraction of annotated parent-task requirements still unanswered after the system claims to have resumed or completed the task.
Interruption response latency	Time from the end of a user interjection to acknowledgement or start of the side-thread answer.
Resumption latency	Time from completion of the detour to resumption of the parent answer, when resumption is expected.
Resource overhead	Additional tokens, memory, tool calls and compute attributable to thread tracking and checkpointing.

7.6 Human evaluation

Automated scoring can assess routing labels, requirement coverage and consistency against structured annotations. Human evaluation is still required for naturalness, whether the resumed answer genuinely follows the earlier explanation, whether contextual bridges are helpful, and whether the assistant resumes at an appropriate time. Evaluators should be blind to the system condition where possible. Inter-annotator agreement should be reported for subjective labels and rubrics.

7.7 Statistical analysis

Report confidence intervals and effect sizes, not only aggregate accuracy. Use paired comparisons when the same conversations are evaluated across systems. Select statistical tests based on the metric distribution and experimental design; for categorical outcomes, appropriate proportion or mixed-effects analyses may be used. Include per-category results because a strong aggregate score

can conceal failures on corrections, multiple active threads or permanent topic changes. Pre-register primary metrics and hypotheses before the confirmatory evaluation where feasible.

8. Discussion: implications and trade-offs

8.1 From transcript memory to task continuity

The core implication is that conversational memory should include the state of unfinished work, not only a record of prior utterances. A transcript can tell a model what was said, but a checkpoint can tell it what remains to be done. This distinction may be useful beyond voice assistants: tutoring systems can resume an interrupted explanation, coding agents can return to a partially explained debugging sequence, research assistants can preserve an unfinished literature analysis, and planning systems can handle quick constraints without discarding a broader plan.

8.2 Full-duplex interaction is necessary but not sufficient

A microphone that remains available during speech can make natural interruption possible, but does not solve thread routing or reasoning continuity. A system can hear the user perfectly and still lose the parent task. Conversely, a text interface can implement much of the thread-management logic without full-duplex audio. The recommended research strategy is therefore to validate thread continuity in text first, then add voice as a separate interaction layer.

8.3 Context integration versus forced relevance

A side question may be tightly connected to the parent task, loosely related, or unrelated. The Mars-temperature example calls for integration because the answer changes habitat design and energy requirements. An unrelated detour should not be artificially attached to the parent answer. The architecture must therefore support both “resume with integration” and “resume without integration”. A relevance classifier that is too permissive can cause confusing digressions; one that is too conservative can fail to exploit useful information.

8.4 Checkpoint granularity

A very compact checkpoint reduces token and memory cost but may omit important assumptions or logical dependencies. A detailed checkpoint can improve continuity while becoming expensive, stale or difficult to validate. The system should store structured, verifiable units: goals, constraints, completed answer sections, cited facts, unresolved requirements and next actions. It should also track checkpoint provenance and timestamps, allowing later corrections to update affected state.

8.5 User control and interruption timing

The system should not infer that every utterance requires the assistant to stop immediately, nor should it insist on finishing a long sentence before listening. “Stop”, “wait”, “I need to correct that” and a new unrelated question have different urgency. A robust interface should expose simple controls and allow the user to configure whether a temporary side question is followed by automatic resumption or by a prompt asking whether to continue.

8.6 Privacy, safety and fairness

Continuous microphone availability creates risks involving bystanders, accidental capture, sensitive speech, speaker attribution and unintended recording. User consent, visible listening state, easy mute, data minimisation, retention limits and secure processing are core system

requirements rather than optional interface details. The system should avoid storing raw audio when structured state is sufficient. Evaluations should include diverse accents, speaking rates, background noise and speech patterns because interruption detection and endpointing may behave unevenly across users.

8.7 Implications for model and interface design

TACA suggests that a conversational agent should expose an explicit notion of "what I am currently working on" and "what is waiting". This can support better transparency: the user may be shown a subtle indication that the main answer is paused and a side question is being handled. Such visibility can make resumption predictable and reduce the risk that the system silently abandons an important task.

9. Limitations and future work

9.1 Current limitations

- No prototype or experimental results are available. All architectural benefits remain hypotheses until implemented and tested.
- The related-work review is focused and should be expanded into a reproducible systematic search across ACL Anthology, arXiv, IEEE Xplore, ACM Digital Library and relevant speech conferences.
- The proposed thread taxonomy may not capture all conversational behaviours, including humour, sarcasm, collaborative storytelling, implicit topic return and culturally specific turn-taking cues.
- The architecture assumes that the system can construct useful checkpoints from generated output; this may fail when an answer is highly exploratory, uncertain or structurally non-linear.
- Automatic detection of user intent is inherently uncertain. Misclassification can lead to lost information, unwanted resumption or inappropriate interruption.
- Voice deployment adds difficult acoustic problems including echo cancellation, overlapping speech, endpointing, speaker attribution and variable network latency.
- Benchmark scenarios may not represent the full diversity of natural conversations. Human evaluation and testing with real users will be required.

9.2 Future research directions

1. Build and test the text-only TACA prototype, beginning with controlled interleaving and explicit answer checkpoints.
2. Develop ICRB with carefully annotated thread graphs, answer-progress labels and no-resume cases; publish data cards and annotation guidelines.
3. Integrate a full-duplex speech stack and measure whether live listening improves interaction without increasing false interruptions or privacy risk.
4. Investigate adaptive checkpoint granularity: sentence-level, section-level and requirement-level checkpoints.
5. Explore hierarchical thread graphs for long-lived projects containing multiple parent tasks and dependent subtasks.
6. Extend the framework to multimodal context such as screenshots, documents, diagrams and tool outputs.

7. In a later study, investigate how humour, sarcasm, social intent and shared conversational history interact with thread continuity. These should be introduced only after the core resumption task is well defined.

10. Conclusion

This paper has proposed a research direction for thread-aware conversational intelligence in large language models. The motivating observation is that natural conversation is not a sequence of isolated, completed turns: people can temporarily branch into a short question, clarify a detail, add new information and then continue an unfinished discussion. A conversational AI system that treats the latest message as a replacement for the previous task risks losing the structure and progress of that earlier response.

The proposed Thread-Aware Conversational Architecture combines streaming interruption handling, intent and thread-relation classification, a graph-structured thread store, task-level answer checkpoints and relevance-aware resumption. Its central design principle is to preserve the unfinished objective and next unresolved response unit, then use information from a side thread only when it legitimately changes or enriches the parent task. The framework distinguishes logical task continuity from persistence of a model's hidden internal state and does not require private chain-of-thought storage.

The paper also identifies important prior work in dialogue segmentation, topic shifts, long-term memory, full-duplex speech, turn-taking and interruption evaluation. These areas establish a strong research foundation and make it necessary to define the proposed contribution carefully. The candidate contribution is the coordinated management of an interrupted assistant answer and its interleaved side threads, including correct resumption and contextual integration. A prototype, benchmark and controlled evaluation are the necessary next steps before effectiveness or novelty can be claimed.

References

[1] Das, S. S. S., Shah, C., Wan, M., Neville, J., Yang, L., Andersen, R., Buscher, G., and Safavi, T. (2024). S3-DST: Structured Open-Domain Dialogue Segmentation and State Tracking in the Era of LLMs. Findings of the Association for Computational Linguistics: ACL 2024, 14996–15014. https://doi.org/10.18653/v1/2024.findings-acl.891

[2] Hwang, Y., Kim, Y., Jang, Y., Bang, J., Bae, H., and Jung, K. (2024). MP2D: An Automated Topic Shift Dialogue Generation Framework Leveraging Knowledge Graphs. Proceedings of EMNLP 2024, 17682–17702. https://doi.org/10.18653/v1/2024.emnlp-main.979

[3] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. (2024). Evaluating Very Long-Term Conversational Memory of LLM Agents. Proceedings of ACL 2024, 13851–13870. https://doi.org/10.18653/v1/2024.acl-long.747

[4] Défossez, A., Mazaré, L., Orsini, M., Royer, A., Pérez, P., Jégou, H., Grave, E., and Zeghidour, N. (2024). Moshi: A Speech-Text Foundation Model for Real-Time Dialogue. arXiv:2410.00037. https://arxiv.org/abs/2410.00037

[5] Arora, S., Lu, Z., Chiu, C.-C., Pang, R., and Watanabe, S. (2025). Talking Turns: Benchmarking Audio Foundation Models on Turn-Taking Dynamics. International Conference on Learning Representations (ICLR 2025). https://arxiv.org/abs/2503.01174

[6] Gosai, A., Vuong, T., Tyagi, U., Li, S., You, W., Bavare, M., Uçar, A., Fang, Z., Jang, B., Liu, B., and He, Y. (2026). Audio MultiChallenge: A Multi-Turn Evaluation of Spoken Dialogue Systems on Natural Human Interaction. Proceedings of ACL 2026, 35740–35770. https://doi.org/10.18653/2026.acl-long.1654

[7] Kuan, S. Y. S., Chiang, C.-H., Lin, G.-T., and Lee, H. (2026). Model Interruption Bench: Evaluating Real-Time Error Correction in Full-Duplex Voice Agents. Proceedings of EMNLP 2026, 53341–53356. https://doi.org/10.18653/2026.emnlp-main.2349

Appendix A. Proposed thread-state schema

The following illustrative schema is intended to make checkpoints inspectable and testable. It is not a final database design.

```
ThreadState {
  thread_id: string,
  parent_thread_id: string | null,
  relation_to_parent: enum,
  goal: string,
  constraints: [string],
  assumptions: [string],
  completed_answer_units: [string],
  unresolved_requirements: [string],
  evidence_refs: [string],
  next_response_unit: string,
  emitted_output_cursor: { generated: int, played: int },
  lifecycle_status: active | suspended | completed | abandoned | needs_clarification,
  resume_policy: auto | ask | user_triggered,
  updated_at: timestamp
}
```

The “played” cursor is important in speech mode: text generated by the model but not yet heard by the user should not automatically be treated as completed conversational content. The schema should also record provenance for facts introduced by tools or external sources.

Appendix B. Benchmark annotation taxonomy

Annotation field	Allowed values / description
parent_thread_id	Identifier of the unfinished task being interrupted, if any.
incoming_intent	Continue; clarification; side question; additional context; correction; topic change; control; ambiguous.
target_thread_id	The thread to which the incoming user event belongs.
thread_relation	Clarifies; adds evidence; depends on; corrects; temporary detour; unrelated; new parent.
resume_expected	Yes; no; conditional; unclear.
resume_point	Next unresolved answer unit or response obligation.
integration_requirement	None; optional; required; prohibited.
completion_criteria	Atomic elements needed to satisfy the parent goal.
error_labels	Wrong thread; lost state; repeated answer; missed constraint; forced link; unwanted resumption; incomplete goal.

Appendix C. High-level interaction algorithm

1. Initialise the parent thread T_parent from the user goal.
2. Build an answer outline and begin streaming the response.
3. Update the checkpoint from content actually delivered to the user.
4. While streaming, listen for user input only in an explicitly enabled live mode.
5. On interruption: pause/cancel output and capture the utterance.
6. Classify intent and select the target thread.
7. If clarification or side question: create/update a child thread and preserve T_parent.
8. If context or correction: update T_parent and re-plan dependent answer units.
9. Answer the selected side thread and record any new evidence with provenance.
10. Decide whether the user expects resumption; ask if confidence is insufficient.
11. If resuming, select the next unresolved unit in T_parent.
12. Integrate side-thread information only when relevant and supported.
13. Generate a concise bridge when useful, then continue the parent answer.
14. Update thread status and checkpoint; repeat until completed or abandoned.

Copyright and permissions

Copyright © 2026 Pranav Patil. All rights reserved.

This manuscript, including its original text, diagrams, and proposed framework, is protected by copyright. You may cite and discuss this work with appropriate attribution. Reproduction, redistribution, or publication of substantial portions requires prior written permission from the author, except for uses permitted under applicable copyright law.